

1. Hello Linux



```
praks@praks: ~  
GNU nano 8.7.1 hello.c  
#include<stdio.h>  
int main()  
{  
    printf("Hello Linux\n");  
    return 0;  
}
```

```
praks@praks:~$ nano hello.c  
praks@praks:~$ gcc hello.c -o hello  
./hello  
Hello Linux
```

2. Read User Name



```
praks@praks: ~  
GNU nano 8.7.1 username.c *  
#include<stdio.h>  
int main()  
{  
    char name[30];  
    printf("Enter your name: ");  
    scanf("%s",name);  
    printf("Welcome %s\n",name);  
    return 0;  
}
```

```
praks@praks:~$ nano username.c  
praks@praks:~$ gcc username.c -o username  
./username  
Enter your name: Prakshitha  
Welcome Prakshitha
```

3. Display Process ID

```
praks@praks: ~  
GNU nano 8.7.1 processid.c *  
#include<stdio.h>  
#include<unistd.h>  
int main()  
{  
    printf("Process ID = %d\n",getpid());  
    return 0;  
}  
  
praks@praks:~$ nano processid.c  
praks@praks:~$ gcc processid.c -o processid  
./processid  
Process ID = 810
```

4. Fork()

```
praks@praks: ~  
GNU nano 8.7.1 fork1.c *  
#include<stdio.h>  
#include<unistd.h>  
int main()  
{  
    fork();  
    printf("Hello\n");  
    return 0;  
}
```

```
praks@praks:~$ nano fork1.c  
praks@praks:~$ gcc fork1.c -o fork1  
./fork1  
Hello  
Hello
```

5. Parent and Child

```
praks@praks: ~
GNU nano 8.7.1 parentchild.c *
#include<stdio.h>
#include<unistd.h>
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else
    {
        printf("I am Parent Process\n");
    }
    return 0;
}
```

```
praks@praks:~$ nano parentchild.c
praks@praks:~$ gcc parentchild.c -o parentchild
./parentchild
I am Parent Process
I am Child Process
```

6. Display parent and Child PID

```
praks@praks: ~
GNU nano 8.7.1 pids.c *
#include<stdio.h>
#include<unistd.h>
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }
    return 0;
}
```

```
praks@praks:~$ nano pids.c
praks@praks:~$ gcc pids.c -o pids
./pids
Parent PID = 906
Child PID = 907
Parent PID = 325
```

7. Wait()

```
praks@praks: ~  
GNU nano 8.7.1 wait1.c *  
#include<stdio.h>  
#include<unistd.h>  
#include<sys/wait.h>  
int main()  
{  
    pid_t pid;  
    pid=fork();  
    if(pid==0)  
    {  
        printf("Child Running\n");  
    }  
    else  
    {  
        wait(NULL);  
        printf("Parent Running After Child\n");  
    }  
    return 0;  
}
```

```
praks@praks:~$ nano wait1.c  
praks@praks:~$ gcc wait1.c -o wait1  
./wait1  
Child Running  
Parent Running After Child  
praks@praks:~$ |
```

8. Exec()

```
praks@praks: ~
GNU nano 8.7.1                                exec1.c *
#include<stdio.h>
#include<unistd.h>
int main()
{
    printf("Before exec()\n");
    execlp("ls", "ls", NULL);
    printf("After exec()\n");
    return 0;
}

praks@praks:~$ nano exec1.c
praks@praks:~$ gcc exec1.c -o exec1
./exec1
Before exec()
Documents  PRAKS  a.txt  data.txt  exec1.c  file1.txt  fork1.c  hello.c  parentchild  pids  praks.c  processid.c  username  wait1
OSLab     a.out  b.txt  exec1   file.txt  fork1     hello   new.txt  parentchild.c  pids.c  processid  shell        username.c  wait1.c
```

9. fork() + exec()

```
praks@praks: ~
GNU nano 8.7.1                                forkexec.c *
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        execlp("date", "date", NULL);
    }
    else
    {
        wait(NULL);
        printf("Child Completed\n");
    }
    return 0;
}
```

```
praks@praks:~$ nano forkexec.c
praks@praks:~$ gcc forkexec.c -o forkexec
./forkexec
Tue Aug  4 17:57:57 UTC 2026
Child Completed
praks@praks:~$ |
```

HISTORY

```

praks@praks:~$ nano hello.c
praks@praks:~$ gcc hello.c -o hello
./hello
Hello Linux
praks@praks:~$ nano username.c
praks@praks:~$ gcc username.c -o username
./username
Enter your name: Prakshitha
Welcome Prakshitha
praks@praks:~$ nano hello.c
praks@praks:~$ nano processid.c
praks@praks:~$ gcc processid.c -o processid
./processid
Process ID = 810
praks@praks:~$ nano fork1.c
praks@praks:~$ gcc fork1.c -o fork1
./fork1
Hello
Hello
praks@praks:~$ nano parentchild.c
praks@praks:~$ gcc parentchild.c -o parentchild
./parentchild
I am Parent Process
I am Child Process
praks@praks:~$ nano pids.c
praks@praks:~$ gcc pids.c -o pids
./pids
Parent PID = 906
Child PID = 907
Parent PID = 325
praks@praks:~$ nano wait1.c
praks@praks:~$ gcc wait1.c -o wait1
./wait1
Child Running
Parent Running After Child
praks@praks:~$ nano execl.c
praks@praks:~$ gcc execl.c -o execl
./execl
Before exec()
Documents  PRAKS  a.txt  data.txt  execl.c  file1.txt  fork1.c  hello.c  parentchild  pids  praks.c  processid.c  username  wait1
OSLab      a.out  b.txt  execl   file.txt  fork1   hello   new.txt  parentchild.c  pids.c  processid  shell      username.c  wait1.c
praks@praks:~$ nano forkexec.c
praks@praks:~$ gcc forkexec.c -o forkexec
./forkexec
Tue Aug  4 17:57:57 UTC 2026
Child Completed
praks@praks:~$ |

```

```
36 nano hello.c
37 gcc hello.c -o hello
38 ./hello
39 nano username.c
40 gcc username.c -o username
41 ./username
42 nano hello.c
43 nano processid.c
44 gcc processid.c -o processid
45 ./processid
46 nano fork1.c
47 gcc fork1.c -o fork1
48 ./fork1
49 nano parentchild.c
50 gcc parentchild.c -o parentchild
51 ./parentchild
52 nano pids.c
53 gcc pids.c -o pids
54 ./pids
55 nano wait1.c
56 gcc wait1.c -o wait1
57 ./wait1
58 nano execl.c
59 gcc execl.c -o execl
60 ./execl
61 nano forkexec.c
62 gcc forkexec.c -o forkexec
63 ./forkexec
64 history
praks@praks:~$ |
```